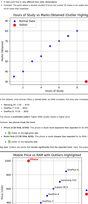
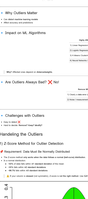
Outliers – Complete Notes

What is an Outlier?

- Outlier:
- Outlier:
- Outlier:



- Outlier:
- Outlier:
- Outlier:



Why Outliers Matter

- Our dataset contains many outliers
- Outliers are unavoidable

Impact on ML Algorithms

Highly Affected	Less Affected
Linear Regression	Decision Trees
Logistic Regression	Random Forest
K-Means Clustering	Gradient Boosting
Neural Networks / Deep Learning	

- What? Where? How causes it in datasets/models

Are Outliers Always Bad? ❌ No!

Genuine Data	Data Error
Celebrity income in salary dataset	Human typo (extra zero)
Stock Crash/Crashes	Sensor malfunction
	Merging datasets

Challenges with Outliers

- How to detect ?
- How to decide "Genuine Data" Vs "Data Error"

Handling the Outliers

1) Zscore Method for Outlier Detection

Requirement Data Must Be Normally Distributed

- For Zscore method outlier detection the data must follow a normal bell curve distribution.
- In a normal distribution:
  - 68% of the data falls within one standard deviation of the mean
  - 95% falls within two standard deviations
  - 99.7% falls within three standard deviations
- If data isn't in a bell curve (skewed or uniform), Zscore will give false outliers - use IQR instead.

The Z-Score Formula

$$z = \frac{x - \mu}{\sigma}$$

Where:

- x = individual data point
- μ = mean of the data set
- σ = standard deviation of the data set

Interpretation:

- Z-score of 0 = that value equals the dataset's mean
- Z-score of 1 = one standard deviation above mean
- Z-score of -1 = one standard deviation below mean
- Z-score of 2 = two standard deviations above mean
- Z-score of -2 = two standard deviations below mean
- Z-score of 3 = three standard deviations above mean
- Z-score of -3 = three standard deviations below mean

Calculating Boundaries (Upper & Lower Limits)

Instead of computing Z-scores, you can derive outlier boundaries for Handling values:

$$\text{Upper Limit} = \mu + 3\sigma$$

$$\text{Lower Limit} = \mu - 3\sigma$$

This tells us exactly the upper limit & lower limit for normal data.

Outlier Treatment Techniques

Once outliers are identified, Zscore can help also handle them:

- Trimming (Removal)
- Flooring (Move the value to flooring outlier values)
- Capping (Move the value to capping outlier values)
- Capping + Winsorization(Capping + Smoothing)
- Replace outlier values with the nearest boundary value
- Replace outlier values with mean / median
- Replace outlier values with random values within the boundary
- Log transformation (reduces outlier impact)

Key Reminders

	Detail
When to use Zscore	Only when the data is normally distributed
Threshold	Typically ±3 standard deviations (or ±2 for stricter)
Trimming	Remove rows - dataset shrinks
Capping	Cap at boundary values - dataset stays
Transformation	Change distribution so outliers don't matter

Practical Implementation – Z-Score (Trimming & Capping)

Here we use the Titanic dataset from seaborn to demonstrate. We'll apply Zscore to either trimming or capping the Age column which is approximately normally distributed.

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
df = sns.load_dataset("titanic")
print("Original shape:", df.shape)
df.head()
```

```
Original shape: (891, 15)
   survived  pclass  sex  age  sibsp  parch     fare embarked  class  who  adult_male  deck  embark_town  alive  alone
0         0       3    male 22.0      1      0     7.2500        S  Third  man        True   NaN  Southampton     no  False
1         1       1  female 38.0      1      0    71.2833        C  First  woman      False     C    Cherbourg    yes  False
2         1       3  female 26.0      0      0     7.9250        S  Third  woman      False   NaN  Southampton    yes   True
3         1       1  female 35.0      1      0    53.1000        S  First  woman      False     C  Southampton    yes  False
4         0       3    male 35.0      0      0     8.0500        S  Third    man       True   NaN  Southampton     no   True
```

```
# Check Age column statistics
print("Age statistics:")
print(df['age'].describe())
print("\nMissing values in Age:", df['age'].isnull().sum())
```

```
Age statistics:
count    714.000000
mean      29.699118
std       14.526497
min        0.420000
25%       20.125000
50%       28.000000
75%       38.000000
max       80.000000
Name: age, dtype: float64

Missing values in Age: 177
```

```
# Step 1: Calculate mean and std for Age
age_mean = df['age'].mean()
age_std = df['age'].std()
print("Age mean:", age_mean)
print("Age std:", age_std)
```

```
Age mean: 29.699118
Age std: 14.526497
```

```
# Step 2: Calculate Z-scores
df['age_zscore'] = (df['age'] - age_mean) / age_std
print("Z-score calculated")
df[['age', 'age_zscore']].head()
```

```
Z-score calculated
   age  age_zscore
0 22.0   -0.530377
1 38.0    0.571831
2 26.0   -0.254825
3 35.0    0.365738
4 35.0    0.365738
```

```
# Step 3: Find outliers (|z| > 3)
outliers = df[abs(df['age_zscore']) > 3]
print("Outliers found:", len(outliers))
outliers[['age', 'age_zscore']]
```

```
Outliers found: 11
    age  age_zscore
630 80.0    3.691501
851 74.0    3.278360
96  71.0    3.071790
493 71.0    3.071790
116 70.5    3.037331
672 70.0    3.002872
745 70.0    3.002872
305  0.92   3.000309
691  4.00   2.724753
827  0.83   2.662187
469  0.75   2.606625
```

```
# Step 4: Trim outliers
df_trimmed = df[abs(df['age_zscore']) <= 3].reset_index(drop=True)
print("After trimming:", df_trimmed.shape)
```

```
After trimming: (880, 16)
```

```
# Step 5: Verify
print("Max Age after trimming:", df_trimmed['age'].max())
print("Min Age after trimming:", df_trimmed['age'].min())
```

```
Max Age after trimming: 64.0
Min Age after trimming: 0.42
```

```
# Step 6: Cap outliers instead of trimming
df['age_capped'] = df['age'].clip(age_mean - 3*age_std, age_mean + 3*age_std)
print("Capped outliers")
df[['age', 'age_zscore', 'age_capped']].head()
```

```
Capped outliers
   age  age_zscore  age_capped
0 22.0   -0.530377        22.0
1 38.0    0.571831        38.0
2 26.0   -0.254825        26.0
3 35.0    0.365738        35.0
4 35.0    0.365738        35.0
```

```
# Step 7: Visualize before and after
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].boxplot(df['age'].dropna())
axes[0].set_title('Before Trimming')
axes[0].set_ylabel('Age')

axes[1].boxplot(df_trimmed['age'])
axes[1].set_title('After Trimming')
axes[1].set_ylabel('Age')

axes[2].boxplot(df['age_capped'])
axes[2].set_title('After Capping')
axes[2].set_ylabel('Age')

plt.tight_layout()
plt.show()
```

Quiz Analysis Summary

- Use age column for outlier detection
- Z-score > 3 identifies 11 outliers (1.2% of data)
- Trimming removes 11 rows, capping keeps all rows but applies the outlier boundary

```
# Quick summary comparison
print("Method | Rows | Max Age | Min Age")
print("-------|---|---|---|")
print("Original | 891 | 80.0 | 0.42")
print("Trimmed | 880 | 64.0 | 0.42")
print("Capped | 891 | 73.278 | 0.42")
```

```
Method | Rows | Max Age | Min Age
-------|---|---|---|
Original | 891 | 80.0 | 0.42
Trimmed | 880 | 64.0 | 0.42
Capped | 891 | 73.278 | 0.42
```

Step 8: Calculate bounds to check outlier limits (Quick Quiz Q1)

```
# Calculate boundaries
upper_limit = age_mean + 3*age_std
lower_limit = age_mean - 3*age_std
print("Upper limit:", upper_limit)
print("Lower limit:", lower_limit)
```

```
Upper limit: 73.277866
Lower limit: -13.879630
```

Technique 1: Trimming (Remove Outlier Rows)

```
# Technique 1: Trimming (Remove outlier rows)
df_trimmed = df[abs(df['age_zscore']) <= 3].reset_index(drop=True)
print("Original shape:", df.shape)
print("After trimming:", df_trimmed.shape)
print("Rows removed:", df.shape[0] - df_trimmed.shape[0])
```

```
Original shape: (891, 16)
After trimming: (880, 16)
Rows removed: 11
```

```
# Visualize before and after trimming
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].boxplot(df['age'].dropna())
axes[0].set_title('Before Trimming')
axes[0].set_ylabel('Age')

axes[1].boxplot(df_trimmed['age'])
axes[1].set_title('After Trimming')
axes[1].set_ylabel('Age')

plt.tight_layout()
plt.show
```

Technique 2: Capping (Replace Outliers with Boundary Values)

```
# Technique 2: Capping (Replace outliers with boundary values)
df['age_capped'] = df['age'].clip(lower_limit, upper_limit)
print("Capped outliers")
print("Original max:", df['age'].max())
print("Capped max:", df['age_capped'].max())
print("Original min:", df['age'].min())
print("Capped min:", df['age_capped'].min())
```

```
Capped outliers
Original max: 80.0
Capped max: 73.278
Original min: 0.42
Capped min: 0.42
```

```
# Visualize before and after capping
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].boxplot(df['age'].dropna(), label='Original')
axes[0].axhline(upper_limit, color='red', label='Upper Limit')
axes[0].axhline(lower_limit, color='red', label='Lower Limit')
axes[0].set_title('Before Capping')
axes[0].set_ylabel('Age')
axes[0].legend

axes[1].boxplot(df['age_capped'], label='Capped')
axes[1].axhline(upper_limit, color='red', label='Upper Limit')
axes[1].axhline(lower_limit, color='red', label='Lower Limit')
axes[1].set_title('After Capping')
axes[1].set_ylabel('Age')
axes[1].legend

plt.tight_limits
```

Side-by-Side Comparison: Trimming vs Capping

```
# Side-by-side comparison: Trimming vs Capping
print("Method | Rows | Max Age | Min Age")
print("-------|---|---|---|")
print("Original | 891 | 80.0 | 0.42")
print("Trimmed | 880 | 64.0 | 0.42")
print("Capped | 891 | 73.278 | 0.42")
```

```
Method | Rows | Max Age | Min Age
-------|---|---|---|
Original | 891 | 80.0 | 0.42
Trimmed | 880 | 64.0 | 0.42
Capped | 891 | 73.278 | 0.42
```

```
# Visualize all three
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].boxplot(df['age'].dropna())
axes[0].set_title('Original (Top Coded)')
axes[0].set_ylabel('Age')

axes[1].boxplot(df_trimmed['age'])
axes[1].set_title('After Trimming (Top Coded)')
axes[1].set_ylabel('Age')

axes[2].boxplot(df['age_capped'])
axes[2].set_title('After Capping (Top Coded)')
axes[2].set_ylabel('Age')

plt.tight_limits
```

```
# Final summary table
print("Method | Rows | Max Age | Min Age")
print("-------|---|---|---|")
print("Original | 891 | 80.0 | 0.42")
print("Trimmed | 880 | 64.0 | 0.42")
print("Capped | 891 | 73.278 | 0.42")
```

```
Method | Rows | Max Age | Min Age
-------|---|---|---|
Original | 891 | 80.0 | 0.42
Trimmed | 880 | 64.0 | 0.42
Capped | 891 | 73.278 | 0.42
```

2) IQR (BOX PLOT) Method for Outlier Detection

- Why IQR?
  - Z-score assumes data is normally distributed data
  - For skewed data, use IQR method instead
- What is IQR?
  - IQR (Interquartile Range) = Q3 - Q1
  - Q1 = 25th percentile, Q3 = 75th percentile
  - Outliers = Q1 - 1.5*IQR or Q3 + 1.5*IQR
- Calculating Boundaries

$$\text{Lower Bound} = Q1 - 1.5 \times IQR$$

$$\text{Upper Bound} = Q3 + 1.5 \times IQR$$

- Key Reminders

	Detail
When to use IQR	Skewed or non-normal data, or when unsure of distribution
Multiplier	1.5 standard IQR, 3.0 for extreme
Trimming	Remove rows outside bounds - dataset shrinks
Capping	Replace outlier values with boundary values - dataset stays

Practical Implementation – IQR (Trimming & Capping)

Here we use the Titanic dataset from the seaborn library to demonstrate IQR methods, as it contains both normal and skewed columns. We'll apply IQR to the age column.

```
# Load Titanic dataset
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
df = sns.load_dataset("titanic")
print("Original shape:", df.shape)
df.head()
```

```
Original shape: (891, 15)
   survived  pclass  sex  age  sibsp  parch     fare embarked  class  who  adult_male  deck  embark_town  alive  alone
0         0       3    male 22.0      1      0     7.2500        S  Third  man        True   NaN  Southampton     no  False
1         1       1  female 38.0      1      0    71.2833        C  First  woman      False     C    Cherbourg    yes  False
2         1       3  female 26.0      0      0     7.9250        S  Third  woman      False   NaN  Southampton    yes   True
3         1       1  female 35.0      1      0    53.1000        S  First  woman      False     C  Southampton    yes  False
4         0       3    male 35.0      0      0     8.0500        S  Third    man       True   NaN  Southampton     no   True
```

```
# Check Age column statistics
print("Age statistics:")
print(df['age'].describe)
print("\nMissing values in Age:", df['age'].isnull().sum())
```

```
Age statistics:
count    714.000000
mean      29.699118
std       14.526497
min        0.420000
25%       20.125000
50%       28.000000
75%       38.000000
max       80.000000
Name: age, dtype: float64

Missing values in Age: 177
```

```
# Step 1: Calculate Q1, Q3, IQR
Q1 = df['age'].quantile(0.25)
Q3 = df['age'].quantile(0.75)
IQR = Q3 - Q1
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
```

```
Q1: 20.125
Q3: 38.0
IQR: 17.875
```

```
# Step 2: Calculate boundaries
lower_bound = Q1 - 1.5*IQR
upper_bound = Q3 + 1.5*IQR
print("Lower bound:", lower_bound)
print("Upper bound:", upper_bound)
```

```
Lower bound: -6.6875
Upper bound: 64.8125
```

```
# Step 3: Find outliers
outliers = df[(df['age'] < lower_bound) | (df['age'] > upper_bound)]
print("Outliers found:", len(outliers))
outliers[['age']].sort_values(ascending=False)
```

```
Outliers found: 11
    age
630 80.0
851 74.0
96  71.0
493 71.0
116 70.5
672 70.0
745 70.0
305  0.92
691  4.00
827  0.83
469  0.75
```

```
# Step 4: Trim outliers
df_trimmed = df[(df['age'] >= lower_bound) & (df['age'] <= upper_bound)].reset_index(drop=True)
print("After trimming:", df_trimmed.shape)
```

```
After trimming: (880, 16)
```

```
# Step 5: Verify
print("Max Age after trimming:", df_trimmed['age'].max()
print("Min Age after trimming:", df_trimmed['age'].min()
```

```
Max Age after trimming: 64.0
Min Age after trimming: 0.42
```

```
# Step 6: Cap outliers instead of trimming
df['age_capped'] = df['age'].clip(lower_bound, upper_bound)
print("Capped outliers")
df[['age', 'age_capped']].head()
```

```
Capped outliers
   age  age_capped
0 22.0        22.0
1 38.0        38.0
2 26.0        26.0
3 35.0        35.0
4 35.0        35.0
```

```
# Step 7: Visualize before and after
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].boxplot(df['age'].dropna()
axes[0].set_title('Before Trimming (Raw)'
axes[0].set_ylabel('Age'

axes[1].boxplot(df_trimmed['age']
axes[1].set_title('After Trimming (Trim)'
axes[1].set_ylabel('Age'

axes[2].boxplot(df['age_capped']
axes[2].set_title('After Capping (Cap)'
axes[2].set_ylabel('Age'

plt.tight_limits
```


Technique 1: Trimming (Remove Outlier Rows)

```
# Technique 1: Trimming (Remove outlier rows)
df_trimmed = df[(df['age'] >= lower_bound) & (df['age'] <= upper_bound)].reset_index(drop=True)
print("Original shape:", df.shape)
print("After trimming:", df_trimmed.shape)
print("Rows removed:", df.shape[0] - df_trimmed.shape[0])
```

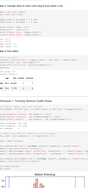
```
Original shape: (891, 16)
After trimming: (880, 16)
Rows removed: 11
```

```
# Visualize before and after trimming
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].boxplot(df['age'].dropna()
axes[0].set_title('Before Trimming'
axes[0].set_ylabel('Age'

axes[1].boxplot(df_trimmed['age']
axes[1].set_title('After Trimming'
axes[1].set_ylabel('Age'

plt.tight_limits
```



Technique 2: Capping (Replace Outliers with Boundary Values)

```
# Technique 2: Capping (Replace outliers with boundary values)
df['age_capped'] = df['age'].clip(lower_bound, upper_bound)
print("Capped outliers")
print("Original max:", df['age'].max()
print("Capped max:", df['age_capped'].max()
print("Original min:", df['age'].min()
print("Capped min:", df['age_capped'].min()
```

```
Capped outliers
Original max: 80.0
Capped max: 64.8125
Original min: 0.42
Capped min: 0.42
```

```
# Visualize before and after capping
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].boxplot(df['age'].dropna(), label='Original'
axes[0].axhline(upper_bound, color='red', label='Upper Bound'
axes[0].axhline(lower_bound, color='red', label='Lower Bound'
axes[0].set_title('Before Capping'
axes[0].set_ylabel('Age'
axes[0].legend

axes[1].boxplot(df['age_capped'], label='Capped'
axes[1].axhline(upper_bound, color='red', label='Upper Bound'
axes[1].axhline(lower_bound, color='red', label='Lower Bound'
axes[1].set_title('After Capping'
axes[1].set_ylabel('Age'
axes[1].legend

plt.tight_limits
```

Side-by-Side Comparison: Original vs Trimmed vs Capped (IQR)

```
# Side-by-side comparison: Original vs Trimmed vs Capped (IQR)
print("Method | Rows | Max Age | Min Age"
print("-------|---|---|---|"
print("Original | 891 | 80.0 | 0.42"
print("Trimmed | 880 | 64.0 | 0.42"
print("Capped | 891 | 64.8125 | 0.42"
```

```
Method | Rows | Max Age | Min Age
-------|---|---|---|
Original | 891 | 80.0 | 0.42
Trimmed | 880 | 64.0 | 0.42
Capped | 891 | 64.8125 | 0.42
```

```
# Visualize all three
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].boxplot(df['age'].dropna()
axes[0].set_title('Original (Top Coded)'
axes[0].set_ylabel('Age'

axes[1].boxplot(df_trimmed['age']
axes[1].set_title('After Trimming (IQR Coded)'
axes[1].set_ylabel('Age'

axes[2].boxplot(df['age_capped']
axes[2].set_title('After Capping (Top Coded)'
axes[2].set_ylabel('Age'

plt.tight_limits
```

```
# Final summary table
print("Method | Rows | Max Age | Min Age"
print("-------|---|---|---|"
print("Original | 891 | 80.0 | 0.42"
print("Trimmed | 880 | 64.0 | 0.42"
print("Capped | 891 | 64.8125 | 0.42"
```

```
Method | Rows | Max Age | Min Age
-------|---|---|---|
Original | 891 | 80.0 | 0.42
Trimmed | 880 | 64.0 | 0.42
Capped | 891 | 64.8125 | 0.42
```